SPAM EMAIL DETECTION

Cybersecurity Mini Project Report

Technology: Python · Flask · scikit-learn · TF-IDF · Naive Bayes

Student Name: Hariharan G K

Register Number: 2503617814921016

Department / College: BE CSE (Cybersecurity) / Sona College of Technology, Salem

Academic Project Report

Spam Email Detection Using Machine Learning

1. Abstract

Spam Email Detection is a web application that classifies text messages as spam or ham (not spam). It uses TF-IDF to convert text into numerical features and a Multinomial Naive Bayes classifier to predict the class. A Flask interface lets a user submit text and view the result. The included sample model is for demonstration only and should be retrained and evaluated with a larger labelled dataset for meaningful results.

2. Problem Statement

Unwanted and deceptive messages can contain misleading offers, suspicious requests, or attempts to trick recipients. A text classification system can help flag messages that resemble known spam patterns for further review.

3. Objectives

• Build a simple web interface for message classification.

• Process message text using TF-IDF.

• Train a Naive Bayes classifier.

• Display the predicted class and estimated class probability.

• Provide a reusable training script for labelled CSV data.

4. Technologies Used

Python, Flask, scikit-learn, TF-IDF, Multinomial Naive Bayes, HTML, and CSS.

5. Modules

1. User interface: accepts message text.

2. Text vectorization: transforms words into numerical TF-IDF features.

3. Classification: predicts spam or ham.

4. Result display: presents the predicted label and model probability.

5. Model training: optionally trains the classifier from a labelled CSV file.

6. Methodology

Collect labelled text → vectorize text using TF-IDF → train a Multinomial Naive Bayes model → accept a new message → transform it using the fitted vectorizer → predict the class → display the result.

7. Sample Test Cases

• Promotional message containing a prize claim: may be classified as spam.

• Ordinary class or meeting message: may be classified as ham.

• Empty input: the app asks the user to enter a message.

• Very long input: the app displays a length warning.

The first two examples are illustrative; actual results depend on the trained model.

8. Limitations

The built-in dataset is very small and cannot provide reliable real-world spam detection. The displayed class probability is not the same as measured model accuracy. The system does not inspect email headers, attachments, sender reputation, or links.

9. Future Enhancements

Use a larger public labelled dataset, evaluate with a held-out test set, report precision/recall/F1-score, analyze URLs and email headers, support multiple languages, and add feedback-based model improvement.

10. Conclusion

This project demonstrates the basic machine-learning workflow for text classification through a simple web application. It provides a foundation for learning about NLP and cybersecurity, while emphasizing that a real anti-spam system requires a larger dataset and proper evaluation.